

Neural Models Lab

Samarth Bhatnagar

September 2026

Contents

Contents

1	Task 1: Understand the Problem Before Coding	2
2	Task 2: Design the Intelligent Agent	3
3	Task 3: Use an LLM to Generate a First Implementation	4
4	Task 4: Execute, test, and diagnose the generated code	5
5	Task 5: Extend the Task and Reflect on the Outcome	7
6	Final Code	9
6.1	Binary XOR Experiment	9
6.2	Symmetry Experiment	11
6.3	Activation Function Experiment	12
6.4	Three-Class Extension	13
7	Reflection Questions	15

1 Task 1: Understand the Problem Before Coding

1. Input Space, Output Space, and Labelled Examples

The input space is

$$\mathcal{X} = \{0, 1\}^2$$

and the output space is

$$\mathcal{Y} = \{0, 1\}.$$

The four labelled examples are

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

2. XOR Points in the x_1, x_2 Plane

The four XOR examples are plotted in the x_1, x_2 plane below.

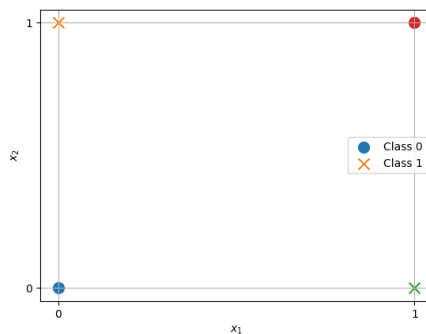


Figure 1: XOR dataset in the x_1, x_2 plane.

3. Why a Straight Decision Boundary Cannot Separate the Classes

The XOR classes are not linearly separable because the class-1 points $(0, 1)$ and $(1, 0)$ lie diagonally opposite the class-0 points $(0, 0)$ and $(1, 1)$. Therefore, no single straight decision boundary can separate the two classes.

4. Prediction for a Single Affine Transformation Followed by a Sigmoid

A single affine transformation followed by a sigmoid can only produce a linear decision boundary. Since XOR is not linearly separable, the model will not be able to classify all four examples correctly. Therefore, I expect the model to reduce the loss to some extent but fail to learn the XOR function perfectly.

Think About It

XOR tests the scientific claim that having more parameters or greater depth alone is not sufficient; the model must have the appropriate nonlinear representation. With only four data points, XOR demonstrates that an affine-only model cannot represent the required decision rule, whereas introducing a nonlinear hidden representation can make the classes separable.

2 Task 2: Design the Intelligent Agent

Model Specification

The baseline neural network consists of

$$2 \text{ inputs} \rightarrow 2 \text{ hidden units} \rightarrow 1 \text{ output.}$$

The hidden layer uses a nonlinear activation function. For the baseline model, sigmoid activation is used. The output layer uses a sigmoid activation, and the model is trained using binary cross-entropy loss with gradient-based optimisation.

1. Why is the Hidden Nonlinearity Scientifically Necessary?

The hidden nonlinearity is necessary because XOR is not linearly separable. A stack of affine transformations without nonlinear activations is equivalent to a single affine transformation and therefore cannot represent the XOR decision boundary. The nonlinear hidden layer allows the network to transform the input into a representation in which the two classes can be separated.

2. Why is Sigmoid Plus Binary Cross-Entropy a Sensible Engineering Pairing?

The target is a binary yes/no prediction, so a sigmoid output produces a value between 0 and 1, which can be interpreted as the predicted probability of the positive class. Binary cross-entropy is appropriate for measuring the difference between these predicted probabilities and the binary target labels.

3. Validation Criteria

The following checks will be used to determine whether successful learning has occurred:

1. The final loss should be substantially lower than the initial loss.
2. All four XOR examples should be classified correctly after thresholding the predicted probabilities at 0.5.
3. The gradients of the model parameters should be nonzero during training, demonstrating that a learning signal is being propagated through the network.
4. The behaviour should be reproducible across repeated runs when the same random seed is used.

Think About It

The hidden units are not assigned explicit target values. Backpropagation determines how the hidden-unit parameters should change by propagating the output loss backwards through the network. The gradients provide each hidden unit with a learning signal, allowing the network to discover useful intermediate representations for solving XOR.

3 Task 3: Use an LLM to Generate a First Implementation

LLM Prompt Used

Generate minimal PyTorch code for the following model and dataset. Do not change the architecture or task. Use the four XOR training examples explicitly, with a 2–2–1 network and sigmoid activation in the hidden layer. Use a sigmoid binary output, or preferably logits with `BCEWithLogitsLoss`. Use random weight initialization and full-batch training for a few thousand lightweight CPU steps. After training, report the final loss, all four probabilities, thresholded labels, and one parameter-gradient tensor. Set a random seed for reproducibility and explain each test in one sentence.

Code Inspection

The forward pass occurs in the model's `forward()` method. The scalar loss is formed using `BCEWithLogitsLoss`. Reverse-mode automatic differentiation is invoked by calling `loss.backward()`, which computes the gradients of the loss with respect to the model parameters. The optimiser updates the parameters when `optimizer.step()` is called.

Changes Made Before Execution

No changes were made to the model architecture or the scientific task. The implementation was checked to ensure that it uses the required 2–2–1 architecture, full-batch XOR training, random initialization, and the specified loss.

Think About It

The architecture, dataset, and mathematical operations can be verified from the code without execution. In particular, the network architecture, activation function, loss function, optimizer, random initialization, and training procedure can be inspected directly. However, the actual learning behaviour, final loss, predicted probabilities, correctness of the four labels, and numerical gradient values require execution and measurement.

4 Task 4: Execute, test, and diagnose the generated code

Part A – Basic Learning Check

The initial loss was

$$L_{\text{initial}} = 0.7482295,$$

while the final loss after training was

$$L_{\text{final}} = 0.00033991.$$

The final predicted probabilities and thresholded predictions were:

(x_1, x_2)	$P(y = 1)$	Predicted label
(0, 0)	0.000308	0
(0, 1)	0.999694	1
(1, 0)	0.999554	1
(1, 1)	0.000299	0

All four XOR examples were classified correctly.

Part B – Backpropagation Check

The quantity `parameter.grad` represents the gradient of the loss with respect to that parameter. For the first-layer weight matrix, it represents

$$\frac{\partial L}{\partial W^{(1)}}.$$

The first-layer weight gradient obtained after the backward pass was

$$\frac{\partial L}{\partial W^{(1)}} = \begin{bmatrix} -0.0040 & -0.0044 \\ 0.0064 & 0.0081 \end{bmatrix}.$$

The corresponding Euclidean norm of the early first-layer gradient was

$$\|\nabla_{W^{(1)}} L\|_2 = 0.0118937.$$

The nonzero gradient provides a learning signal indicating how the first-layer weights should change to reduce the loss.

Since the loss is computed as the mean over the four training examples, its gradient is also the average of the gradients contributed by the individual examples:

$$\frac{\partial L}{\partial W^{(1)}} = \frac{1}{4} \sum_{i=1}^4 \frac{\partial L_i}{\partial W^{(1)}}.$$

Part C – Symmetry Experiment

When all trainable parameters were initialized to zero, the two rows of the hidden-layer weight matrix remained identical over the observed training steps. In fact, they remained zero:

$$W^{(1)} = \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}.$$

The two hidden units therefore compute exactly the same output. Because they are identical, they also receive the same gradient during backpropagation. Consequently, the symmetry is not broken and the two hidden units cannot learn distinct features.

Part D – Activation Experiment

Hidden activation	Final loss	4/4 correct?	$\ \nabla_{W^{(1)}} L\ _2$ (early)
Sigmoid	0.000340	Yes	0.011894
Tanh	0.346596	No	0.028684
ReLU	0.346582	No	0.082002

Table 1: Results of the activation experiment.

In this experiment, the sigmoid network successfully classified all four XOR examples, whereas the tanh and ReLU networks did not reach four correct classifications under the same training settings. The early gradient norms were different for the three activations, showing that the choice of activation affects the gradient signal. These observations describe the behaviour of this particular experiment and do not establish that one activation is universally better than the others.

Think About It

The two mechanisms can be distinguished by inspecting the pre-activation values and the corresponding activations. For a sigmoid unit, a pre-activation with a very large positive or negative magnitude produces a saturated sigmoid output near 1 or 0, whose derivative is close to zero. For a ReLU unit, a negative pre-activation produces an output of zero and a derivative of zero. Thus, examining the pre-activation values allows us to distinguish sigmoid saturation from a ReLU unit being in its inactive region.

5 Task 5: Extend the Task and Reflect on the Outcome

Modified Output Design

The binary XOR task was extended to three classes:

Class	Meaning	Input
0	Both sensors inactive	(0, 0)
1	Sensors disagree	(0, 1), (1, 0)
2	Both sensors active	(1, 1)

The hidden representation remains two-dimensional, while the output layer is changed from one output to three logits. Thus, the final layer has weight matrix

$$W^{(2)} \in R^{3 \times 2}.$$

Each input therefore produces three logits, one for each class. The model is trained using multiclass cross-entropy loss.

Predictions

The predicted class probabilities were approximately

Input	$P(y = 0)$	$P(y = 1)$	$P(y = 2)$
(0, 0)	0.99972	0.000279	0.000000003
(0, 1)	0.000112	0.99979	0.000098
(1, 0)	0.000110	0.99979	0.000100
(1, 1)	0.000000009	0.000286	0.99971

The resulting predicted classes are

$$(0, 0) \rightarrow 0, \quad (0, 1) \rightarrow 1, \quad (1, 0) \rightarrow 1, \quad (1, 1) \rightarrow 2.$$

For the input (0, 0), the softmax probability vector was

$$[0.99972, 0.000279, 0.000000003],$$

whose components sum to approximately

$$1.000000.$$

Why Softmax Probabilities Sum to One

For logits z_1, z_2, z_3 , the softmax probability for class i is

$$p_i = \frac{e^{z_i}}{\sum_{j=1}^3 e^{z_j}}.$$

Therefore,

$$\sum_{i=1}^3 p_i = \frac{\sum_{i=1}^3 e^{z_i}}{\sum_{j=1}^3 e^{z_j}} = 1.$$

Why the Logit Gradient Has the Form $p - y$

For a one-hot target vector y and softmax probabilities p , the cross-entropy loss is

$$L = - \sum_i y_i \log p_i.$$

Differentiating the softmax cross-entropy with respect to the logits gives

$$\frac{\partial L}{\partial z_i} = p_i - y_i.$$

Thus, the gradient increases the relative contribution of the correct class when its predicted probability is too small and decreases the contribution of classes whose predicted probabilities are too large.

Optional Numerical Stability Check

Adding the same constant c to every logit does not change the softmax probabilities:

$$\frac{e^{z_i+c}}{\sum_j e^{z_j+c}} = \frac{e^c e^{z_i}}{e^c \sum_j e^{z_j}} = \frac{e^{z_i}}{\sum_j e^{z_j}}.$$

Therefore, adding 100 to all three logits produced the same probability vector up to floating-point roundoff. In practice, stable softmax implementations subtract the maximum logit before exponentiating. This prevents very large exponentials and reduces the risk of numerical overflow.

Think About It

The mathematical structure of multiclass next-token prediction remains the same: the model produces one logit for each possible class, applies softmax to obtain a probability distribution, and uses cross-entropy during training. The softmax and cross-entropy gradients retain the form $p - y$.

For a language model, however, the number of classes becomes the vocabulary size, which can be tens of thousands. Consequently, the output layer and surrounding architecture become much larger, and the model requires substantially more computation and memory.

6 Final Code

The complete executable implementation used for the experiments is provided below. The same code was also executed in the accompanying Jupyter/Google Colab notebook.

6.1 Binary XOR Experiment

The following code implements the baseline 2–2–1 neural network, trains it on the four XOR examples, and reports the final loss, probabilities, predictions, and an early first-layer gradient.

```
1 import torch
2 import torch.nn as nn
3
4 torch.manual_seed(42)
5
6 # XOR dataset
7 X = torch.tensor([
8     [0.0, 0.0],
9     [0.0, 1.0],
10    [1.0, 0.0],
11    [1.0, 1.0]
12 ])
13
14 y = torch.tensor([
15     [0.0],
16     [1.0],
17     [1.0],
18     [0.0]
19 ])
20
21 # 2 -> 2 -> 1 network
22 class XORNet(nn.Module):
23     def __init__(self, hidden_activation="sigmoid"):
24         super().__init__()
25
26         self.fc1 = nn.Linear(2, 2)
27         self.fc2 = nn.Linear(2, 1)
28
29         if hidden_activation == "sigmoid":
```

```

30         self.activation = nn.Sigmoid()
31     elif hidden_activation == "tanh":
32         self.activation = nn.Tanh()
33     elif hidden_activation == "relu":
34         self.activation = nn.ReLU()
35     else:
36         raise ValueError("Unknown activation")
37
38     def forward(self, x):
39         h = self.activation(self.fc1(x))
40         return self.fc2(h)
41
42
43 model = XORNet(hidden_activation="sigmoid")
44
45 criterion = nn.BCEWithLogitsLoss()
46 optimizer = torch.optim.Adam(model.parameters(), lr=0.05)
47
48 # Initial loss
49 with torch.no_grad():
50     initial_loss = criterion(model(X), y).item()
51
52 # Capture an early gradient
53 optimizer.zero_grad()
54 loss = criterion(model(X), y)
55 loss.backward()
56
57 early_gradient = model.fc1.weight.grad.detach().clone()
58 early_gradient_norm = torch.norm(early_gradient).item()
59
60 optimizer.step()
61
62 # Continue training
63 for step in range(3000):
64     optimizer.zero_grad()
65
66     logits = model(X)
67     loss = criterion(logits, y)
68
69     loss.backward()
70     optimizer.step()
71
72 # Final results
73 with torch.no_grad():
74     logits = model(X)
75     probabilities = torch.sigmoid(logits)
76     predictions = (probabilities >= 0.5).float()
77
78 final_loss = criterion(model(X), y).item()
79
80 print("Initial loss:", initial_loss)
81 print("Final loss:", final_loss)
82
83 print("\nFinal probabilities:")
84 for i in range(len(X)):
85     print(
86         X[i].tolist(),

```

```

87         "->",
88         f"{probabilities[i].item():.6f}"
89     )
90
91     print("\nThresholded predictions:")
92     for i in range(len(X)):
93         print(
94             X[i].tolist(),
95             "->",
96             int(predictions[i].item())
97         )
98
99     print("\nFirst-layer gradient:")
100    print(early_gradient)
101
102    print("\nEarly gradient norm:")
103    print(early_gradient_norm)

```

Listing 1: Binary XOR experiment

6.2 Symmetry Experiment

The following experiment initializes all weights and biases to zero and examines the hidden-layer weights during training.

```

1  print("--- Symmetry Experiment ---")
2
3  torch.manual_seed(42)
4
5  symmetry_model = XORNet(hidden_activation="sigmoid")
6
7  # Set every parameter to zero
8  with torch.no_grad():
9      for parameter in symmetry_model.parameters():
10         parameter.zero_()
11
12  symmetry_optimizer = torch.optim.Adam(
13      symmetry_model.parameters(),
14      lr=0.05
15  )
16
17  for step in range(6):
18
19      symmetry_optimizer.zero_grad()
20
21      logits = symmetry_model(X)
22      loss = criterion(logits, y)
23
24      loss.backward()
25      symmetry_optimizer.step()
26
27      print(f"\nStep {step}:")
28      print("Hidden weight matrix:")
29      print(symmetry_model.fc1.weight.detach())
30
31      print("Hidden bias:")

```

```
32 print(symmetry_model.fc1.bias.detach())
```

Listing 2: Symmetry experiment with zero initialization

6.3 Activation Function Experiment

The same architecture is trained using three different hidden activations: sigmoid, tanh, and ReLU. All other experimental settings are kept unchanged.

```
1 print("--- Activation Experiment ---")
2
3 activations = ["sigmoid", "tanh", "relu"]
4
5 for activation in activations:
6
7     torch.manual_seed(42)
8
9     model = XORNet(hidden_activation=activation)
10
11     optimizer = torch.optim.Adam(
12         model.parameters(),
13         lr=0.05
14     )
15
16     # Initial loss
17     with torch.no_grad():
18         initial_loss = criterion(model(X), y).item()
19
20     # Early gradient
21     optimizer.zero_grad()
22
23     loss = criterion(model(X), y)
24     loss.backward()
25
26     early_norm = torch.norm(
27         model.fc1.weight.grad
28     ).item()
29
30     optimizer.step()
31
32     # Training
33     for step in range(3000):
34
35         optimizer.zero_grad()
36
37         logits = model(X)
38         loss = criterion(logits, y)
39
40         loss.backward()
41         optimizer.step()
42
43     # Final evaluation
44     with torch.no_grad():
45         logits = model(X)
46         probabilities = torch.sigmoid(logits)
47         predictions = (probabilities >= 0.5).float()
```

```

48     final_loss = criterion(model(X), y).item()
49
50     correct = int(
51         (predictions == y).sum().item()
52     )
53
54     print(f"\nActivation: {activation}")
55     print(f"Initial loss: {initial_loss:.6f}")
56     print(f"Final loss: {final_loss:.6f}")
57     print(f"Correct: {correct}/4")
58     print(f"Early gradient norm: {early_norm:.6f}")
59
60     print("Probabilities:")
61     print(probabilities.squeeze().tolist())
62
63     print("Predictions:")
64     print(predictions.squeeze().int().tolist())
65

```

Listing 3: Comparison of hidden activation functions

6.4 Three-Class Extension

The binary XOR problem is extended to three classes:

- Class 0: both sensors inactive, (0,0)
- Class 1: sensors disagree, (0,1) and (1,0)
- Class 2: both sensors active, (1,1)

The output layer therefore produces three logits, and cross-entropy loss is used for multiclass classification.

```

1  print("--- Three-Class Extension ---")
2
3  torch.manual_seed(42)
4
5  # Three-class labels:
6  # (0,0) -> class 0
7  # (0,1) -> class 1
8  # (1,0) -> class 1
9  # (1,1) -> class 2
10
11 X3 = torch.tensor([
12     [0.0, 0.0],
13     [0.0, 1.0],
14     [1.0, 0.0],
15     [1.0, 1.0]
16 ])
17
18 y3 = torch.tensor([
19     0,
20     1,
21     1,

```

```

22     2
23 ])
24
25
26 class ThreeClassNet(nn.Module):
27
28     def __init__(self):
29         super().__init__()
30
31         self.fc1 = nn.Linear(2, 2)
32         self.fc2 = nn.Linear(2, 3)
33
34     def forward(self, x):
35
36         hidden = torch.sigmoid(
37             self.fc1(x)
38         )
39
40         return self.fc2(hidden)
41
42
43 model3 = ThreeClassNet()
44
45 criterion3 = nn.CrossEntropyLoss()
46
47 optimizer3 = torch.optim.Adam(
48     model3.parameters(),
49     lr=0.05
50 )
51
52 for step in range(3000):
53
54     optimizer3.zero_grad()
55
56     logits = model3(X3)
57     loss = criterion3(logits, y3)
58
59     loss.backward()
60     optimizer3.step()
61
62
63 with torch.no_grad():
64
65     logits = model3(X3)
66
67     probabilities = torch.softmax(
68         logits,
69         dim=1
70     )
71
72     predictions = torch.argmax(
73         probabilities,
74         dim=1
75     )
76
77
78 print("Final loss:", criterion3(logits, y3).item())

```

```

79
80 print("\nClass probabilities:")
81
82 for i in range(len(X3)):
83
84     print(
85         X3[i].tolist(),
86         "->",
87         probabilities[i].tolist()
88     )
89
90 print("\nPredictions:")
91
92 for i in range(len(X3)):
93
94     print(
95         X3[i].tolist(),
96         "->",
97         predictions[i].item()
98     )
99
100 # Verify that softmax probabilities sum to one
101 print("\nFirst softmax vector:")
102 print(probabilities[0])
103
104 print("Sum:")
105 print(probabilities[0].sum().item())

```

Listing 4: Three-class extension of the XOR problem

7 Reflection Questions

1. What did the XOR experiment demonstrate about the difference between depth and nonlinearity?

The XOR experiment demonstrated that depth alone does not make a network capable of representing XOR. A stack of affine layers without nonlinear activations collapses into a single affine transformation, so adding depth without nonlinearity does not solve the linear separability problem. A nonlinear hidden layer provides the required representation.

2. In your successful run, what evidence showed that back-propagation supplied a useful learning signal rather than merely a nonzero gradient?

The loss decreased substantially during training, and the final predictions correctly classified all four XOR examples. Thus, the gradients were not merely nonzero; they led to parameter updates that improved the model's predictions and reduced the loss.

3. Why did identical/zero weight initialisation prevent the two hidden units from learning distinct features?

With identical or zero initialization, the two hidden units compute the same output. Since they receive the same inputs and produce the same output, back-propagation gives them the same gradients. Consequently, their symmetry is preserved and they cannot learn distinct features.

4. How did changing the hidden activation affect the gradient you observed? Distinguish the scientific explanation from the engineering observation.

The activation function changes the derivatives through which gradients are propagated. A sigmoid unit can have a very small derivative when it is saturated, while a ReLU unit has zero derivative when its pre-activation is negative. In the experiment, the three activations produced different early gradient norms and different final learning behaviour. These are observations specific to the chosen architecture, initialization, and training settings and do not establish that one activation is universally better.

5. Why must the output layer and loss be selected together according to the task?

The output layer and loss function must match the structure of the prediction task. For binary classification, a sigmoid output with binary cross-entropy is appropriate because the output represents the probability of one of two classes. For multiclass classification, multiple logits with softmax probabilities and multiclass cross-entropy are appropriate.

6. Give one example where the LLM improved your engineering productivity and one example where human verification was essential.

The LLM improved engineering productivity by providing a concise first implementation of the specified PyTorch model, including the model, loss, optimizer, and training loop.

Human verification was essential to check that the generated code actually implemented the required 2–2–1 architecture, used the correct XOR dataset and loss, and produced the required experimental evidence after execution.

7. Which tests in this laboratory would you keep if the model were scaled up, and which would become too expensive?

I would keep practical tests such as monitoring the training and validation loss, checking predictions on representative examples, inspecting gradient norms, and testing for reproducibility. These tests remain useful when the model is scaled up.

Exhaustive finite-difference gradient checks would become too expensive for a large model because they require evaluating the loss repeatedly for many individual parameters. Therefore, such checks would be more appropriate for small implementations than for large-scale models.